

KernelLens v1.1.9: Automated Production Plugin Synthesis, Dynamic Dual-Pass Tracing, and Hardware Alignment Guards for OpenAI Triton Kernels in Enterprise C++ Inference Engines

A Technical Architecture, 12-Step Compiler Formalization, TensorRT 10.16 Integration, and Empirical Verification on Blackwell SOTA LLM Operators

Antigravity AI Engineering Team
Advanced Compiler & Deep Learning Systems Group
Google DeepMind Team

September 20, 2026

Abstract

Writing custom high-performance GPU kernels using OpenAI Triton has become the dominant paradigm for authoring state-of-the-art deep learning operators. However, deploying models containing custom Python `@triton.jit` kernels into production enterprise C++ inference runtimes—specifically Microsoft ONNX Runtime (ORT) and NVIDIA TensorRT (TRT) 10.x—presents severe architectural barriers. Standard ONNX graph export mechanisms cannot trace imperative Python Triton calls, dynamic launch grid functions, implicit layout semantics, in-place memory mutability, dynamic runtime host scalars, or low-level PTX entry parameter signatures.

In this paper, we present **KernelLens v1.1.9**, an end-to-end automated compilation, C++/CUDA code synthesis, and zero-copy runtime framework. KernelLens bridges arbitrary PyTorch models containing custom Triton kernels directly to high-throughput ONNX Runtime and TensorRT custom C++ plugins with zero manual C++ development. We present an exhaustive 12-step technical formalization of KernelLens’s core compiler pipeline: (1) User Model & Kernel Signature Inspection; (2) Target Engine Configuration; (3) Pass 1 Concrete Eager GPU Execution & PTX Artifact Interception; (4) Pass 2 Symbolic FX Proxy Tracing & SymPy Grid Expression Extraction; (5) Static AST Inspection of Main & Recursive Nested Helper Functions; (6) Manifest Validation; (7) Base ONNX Export & Domain Custom Node Injection (`triton_custom::<name>`); (8) Host-Side Dynamic Scalar Binding (`OrtMemTypeCPUInput`); (9) Automated C++/CUDA Code Generation (`ort_gen.py`, `trt_gen.py`); (10) 16-Byte Hardware Memory Alignment Guard Insertion (`Address (mod 16) == 0`); (11) Native `nvcc/g++` Shared Library Compilation (`.so`); and (12) Dynamic Plugin Registration with Zero-Copy VRAM IO Binding.

We detail key architectural innovations in v1.1.9, including: (a) TensorRT 10.16 C++ plugin dynamic creators with explicit symbol registration (`register_triton_plugins_explicit`) enforcing `kLINEAR` tensor formats; (b) Automated Blackwell (`sm_120a`) PTX 9.0 ISA version clamping (`.version 9.3 → .version 9.0`) to satisfy CUDA 13.0 driver JIT require-

ments; (c) Primary CUDA context retention (`cuDevicePrimaryCtxRetain`) to prevent multi-threaded context detachment; and (d) Executable stack security clearance (`patchelf`) for Linux 7.0 kernels. We evaluate KernelLens across state-of-the-art research model kernels from **LLaMA 3**, **Mistral**, **Qwen 2.5**, **PaLM**, and **Liger-Kernel** (Fused RMSNorm, SwiGLU, Rotary Position Embeddings, and Fused Cross-Entropy Loss). KernelLens achieves **exact floating-point numerical parity** (MaxDiff = 0.00e + 00) against eager Triton execution while enabling production-grade C++ inference deployment.

Contents

1	Introduction	4
2	Related Work	5
2.1	PyTorch Compiler & Inductor	5
2.2	ONNX & Enterprise Inference Runtimes	5
2.3	Triton-Based LLM Kernel Libraries	5
3	Anatomy of a Triton Kernel & Parameter Taxonomy	5
3.1	What a Triton Kernel Requires: Formal Parameter Taxonomy	5
4	Complete 12-Step KernelLens Compiler Architecture	6
4.1	Detailed Breakdown of the 12 Pipeline Steps	7
4.1.1	Step 1: User Module & Kernel Definition	7
4.1.2	Step 2: Compiler Invocation (<code>kl.compile</code>)	7
4.1.3	Step 3: Pass 1 – Concrete Eager Execution & Artifact Interception (<code>tracer.py</code>)	7
4.1.4	Step 4: Pass 2 – Symbolic FX Proxy Tracing & SymPy Grid Extraction (<code>tracer.py</code>)	7
4.1.5	Step 5: Static AST Inspection of Main & Nested Helper Functions (<code>ast_analyzer.py</code>)	7
4.1.6	Step 6: Manifest Assembly & Validation (<code>manifest.py</code>)	8
4.1.7	Step 7: Base ONNX Graph Export & Domain Custom Node Injection (<code>onnx_exporter.py</code>)	8
4.1.8	Step 8: Dynamic Host-Side CPU Scalar Binding (<code>OrtMemTypeCPUInput</code>)	8
4.1.9	Step 9: Automated C++/CUDA Code Generation (<code>ort_gen.py</code> , <code>trt_gen.py</code>)	8
4.1.10	Step 10: 16-Byte Hardware Memory Alignment Guard Insertion (<code>builder.py</code>)	9
4.1.11	Step 11: Native C++ Shared Library Compilation & PTX ISA Clamping (<code>builder.py</code>)	9
4.1.12	Step 12: Dynamic Plugin Registration & Zero-Copy Runtime Execution (<code>engine.py</code>)	9
5	16-Byte Memory Alignment Guards & Zero-Copy Runtime	10
5.1	128-Bit SIMD Memory Vectorization in GPU Hardware	10
5.1.1	Mechanics of Hardware Alignment Traps (CUDA Error 716)	10
5.2	Automated C++ Alignment Guard & Dynamic Staging Engine	10
5.3	Zero-Copy Runtime IO Binding Engine (<code>engine.py</code>)	11
5.3.1	1. ONNX Runtime IO Binding	11
5.3.2	2. TensorRT Zero-Copy Pointer Registration	12
6	Experiments & Empirical Results	12
6.1	Benchmark Experimental Setup	12
6.2	Numerical Parity & Accuracy Results	12
6.3	Empirical GPU Execution Latency Benchmark	13

6.4	FP8 & Sub-Byte (INT4/INT8) Quantization Evaluation on NVIDIA RTX 5060 Ti GPU	13
7	Future Work	15
8	Conclusion	15

1 Introduction

Deep learning systems increasingly rely on domain-specific GPU kernels to achieve peak computational efficiency, reduce VRAM bandwidth demands, and fuse multi-stage attention and normalization blocks. OpenAI Triton [1] has established itself as the standard programming model for authoring custom GPU kernels in Python, providing C/CUDA-like execution control with Python syntax.

Despite Triton’s widespread adoption in model authoring and training, a major infrastructure friction point emerges when transitioning models to production deployment. Enterprise deployment pipelines standardly deploy models within specialized C++ execution engines, such as **Microsoft ONNX Runtime (ORT)** [9] and **NVIDIA TensorRT (TRT)** [8], to achieve low latency, stream concurrency, and hardware-specific kernel graph optimizations.

Deploying PyTorch models containing custom `@triton.jit` kernels directly into ONNX Runtime or TensorRT fails due to six critical architectural gaps:

1. **ONNX Exporter Tracing Breakdown:** Standard PyTorch ONNX export utilities (`torch.onnx.export`) operate on TorchScript or TorchDynamo IRs representing native PyTorch C++ operators (`at::native`). Imperative Python calls to `triton.JITFunction` instances are opaque to PyTorch’s trace backend, leading to immediate tracing failure or missing graph nodes.
2. **Dynamic Launch Grid Resolution:** Triton launch grids are defined via dynamic Python lambda functions $\text{grid}(x, y, z) = (g_x(s_i), g_y(s_i), g_z(s_i))$ dependent on runtime tensor dimensions $s_0, s_1, \dots$. Standard graph representations cannot symbolize or execute arbitrary Python grid lambdas inside standalone C++ engines.
3. **PTX Entry Signature Mismatch:** The Triton LLVM backend compiles Python source code directly into Parallel Thread Execution (PTX) assembly code. The resulting PTX entry declarations (`.entry <kernel> (.param .u64 p0, ...)`) re-order parameters, drop unused arguments, and enforce specific bitwidths. Native C++ plugin wrappers must dynamically align host argument pointers to match PTX signatures exactly.
4. **Nested Code & In-Place Mutability Analysis:** Research kernels frequently delegate memory store and load operations to nested Python helper functions or perform in-place buffer mutations ($\text{ptr}_{\text{out}} \equiv \text{ptr}_{\text{in}}$). Static compilers fail to track nested mutation semantics, producing corrupted outputs or invalid graph topologies.
5. **Dynamic Runtime Host Scalar Arguments:** Practical models pass scalar arguments (e.g., scaling factors α , soft-cap limits τ) that change per forward pass. C++ execution engines must accept host-side CPU scalar arguments without triggering costly kernel recompilation.
6. **16-Byte Hardware Memory Alignment Bounds:** Vectorized GPU memory instructions (`ld.global.v4.b32`) strictly require starting VRAM pointers to satisfy 16-byte alignment ($\text{Address} \bmod 16 == 0$). Standard inference allocators may provide sub-aligned memory buffers, causing hardware misaligned address exceptions (`CUDA error 716`).

To bridge this gap, we present **KernelLens**, a comprehensive compilation and runtime synthesis system. KernelLens allows machine learning engineers to write custom PyTorch Triton kernels in pure Python, compile the entire model into optimized ONNX computational graphs paired with C++ custom operator shared libraries (`.so`), and execute them inside ONNX Runtime and TensorRT with zero manual C++ programming.

2 Related Work

2.1 PyTorch Compiler & Inductor

PyTorch 2.0 introduced `torch.compile` powered by TorchInductor [6], which generates Triton code for PyTorch eager graphs. While TorchInductor optimizes PyTorch eager execution, it operates inside a Python process and relies on PyTorch C++ runtime dependencies. It does not provide standalone, self-contained ONNX graph representations or C++ plugin libraries suitable for direct integration into non-Python C++ production servers.

2.2 ONNX & Enterprise Inference Runtimes

Microsoft ONNX Runtime [9] and NVIDIA TensorRT [8] provide production C++ inference execution. Both runtimes support C++ Custom Operator APIs (`OrtCustomOp`, `nvinfer1::IPluginV2DynamicExt`). However, authoring custom plugins manually requires deep knowledge of CUDA C++, memory alignment rules, ONNX schema registration, and TensorRT engine serialization. KernelLens automates the end-to-end generation of these C++ custom plugins directly from Triton JIT artifacts.

2.3 Triton-Based LLM Kernel Libraries

Recent research projects such as **Liger-Kernel** [3], **FlashAttention** [5], and **AOTriton** focus on manually crafting high-throughput Triton kernels for large language models (LLMs). KernelLens complements these libraries by providing the infrastructure layer that automatically compiles any custom Triton kernel into production ONNX Runtime and TensorRT C++ plugins.

3 Anatomy of a Triton Kernel & Parameter Taxonomy

3.1 What a Triton Kernel Requires: Formal Parameter Taxonomy

To understand how KernelLens automatically compiles Triton kernels, we first formalize the exact structural requirements of a `@triton.jit` function. A Triton kernel definition consists of seven distinct parameter categories:

$$\mathcal{K}_{\text{triton}} = \langle \mathbf{P}_{\text{in}}, \mathbf{P}_{\text{out}}, \mathbf{P}_{\text{inplace}}, \mathbf{S}, \mathbf{D}, \mathbf{C}, \mathbf{G} \rangle \quad (1)$$

1. **Input Tensor Pointers ($\mathbf{P}_{\text{in}}$):** Base GPU VRAM memory addresses pointing to read-only input tensors (e.g., `x_ptr`, `weight_ptr`). In PTX assembly, these are represented as 64-bit unsigned integers (`.param .u64`).
2. **Output Tensor Pointers ($\mathbf{P}_{\text{out}}$):** Base GPU VRAM memory addresses pointing to write-only destination buffers (e.g., `out_ptr`).
3. **In-Place Mutated Pointers ($\mathbf{P}_{\text{inplace}}$):** Base memory addresses that are simultaneously read from and written to during kernel execution (e.g., `x_ptr` modified in-place).
4. **Spatial and Channel Strides ($\mathbf{S}$):** Integer parameters specifying memory byte or element strides along each dimension of multi-dimensional tensors (e.g., `stride_x_b`, `stride_x_h`). These allow Triton kernels to handle arbitrary non-contiguous tensor layouts.
5. **Dynamic Runtime Host Scalars ($\mathbf{D}$):** Primitive scalar values passed dynamically per forward execution pass, such as scaling parameters α , learning rates, temperature factors τ , or variance stabilization parameters eps.

6. **Compile-Time Constants (C):** Parameters decorated with `tl.constexpr` (e.g., `BLOCK_SIZE`, `num_warps`, `num_stages`). These are evaluated at compile time and baked into the compiled PTX assembly bytecode.
7. **Dynamic Launch Grid (G):** A 1D, 2D, or 3D launching tuple (g_x, g_y, g_z) computed as a function of runtime tensor dimensions and block sizes:

$$g_x = f_x(d_0, d_1, \dots, \mathbf{C}), \quad g_y = f_y(d_0, d_1, \dots, \mathbf{C}), \quad g_z = f_z(d_0, d_1, \dots, \mathbf{C}) \quad (2)$$

—

4 Complete 12-Step KernelLens Compiler Architecture

KernelLens automates the transformation from PyTorch Triton models to production C++ plugins across 12 explicit steps.

EXCALIDRAW DIAGRAM SPECIFICATION: Complete 12-Step KernelLens Architecture Schema

Use this 12-step layout to draw Figure 1 in Excalidraw:

1. **Step 1: User Module & Kernel Definition** → PyTorch `nn.Module` containing `@triton.jit` function.
2. **Step 2: Compiler Invocation** → Call `kl.compile(model, inputs, backends=["onnx", "tensorrt"])`.
3. **Step 3: Pass 1 Concrete Eager Execution** → Native GPU eager run; hook Triton compilation cache → Extract PTX bytecode, S_{mem} , concrete grid (g_x, g_y, g_z) .
4. **Step 4: Pass 2 Symbolic FX Proxy Tracing** → PyTorch FX proxy objects → Trace grid lambda with SymPy → SymPy AST grid strings `("s0*s1+127)//128")` with control flow fallbacks.
5. **Step 5: Static AST Inspection** → Parse main kernel & recursive nested helper functions (`_helper_store`) → Classify Read/Write/In-Place aliasing.
6. **Step 6: Manifest Assembly** → Build validated `KernelManifest` combining PTX, launch expressions, data types, and strides.
7. **Step 7: Base ONNX Graph Export** → Register `TritonGlobalONNXExporter` hook → Inject custom ONNX node (`triton_custom::<name>`).
8. **Step 8: Dynamic Host Scalar Binding** → Configure `OrtMemTypeCPUInput` attributes for runtime CPU scalars (α, τ) .
9. **Step 9: Automated C++/CUDA Code Generation** → Synthesize `register_ops.cpp`, `<kernel>Op.cu`, and `<kernel>Op.h` (`ort_gen.py`, `trt_gen.py`).
10. **Step 10: 16-Byte Memory Alignment Guard Embedding** → Inject runtime VRAM address guards `(Address (mod 16) == 0)` with dynamic staging fallback.
11. **Step 11: Native Shared Library Compilation** → Query CUDA arch (`sm_80`) → Compile via `nvcc/g++` → Generate `libtriton_ort_plugins.so`.
12. **Step 12: Zero-Copy Runtime Execution** → Dynamic C++ `ctypes.CDLL` loading → Bind PyTorch VRAM pointers directly via `IOBinding` (`MaxDiff = 0.00e + 00`).

4.1 Detailed Breakdown of the 12 Pipeline Steps

4.1.1 Step 1: User Module & Kernel Definition

The user defines a PyTorch `nn.Module` invoking one or more custom `@triton.jit` functions with pointer arguments, tensor strides, dynamic runtime scalars, and `tl.constexpr` constants.

4.1.2 Step 2: Compiler Invocation (`kl.compile`)

The user calls `kl.compile(model, sample_inputs, name="<name>", backends=["onnx", "tensorrt"])`. KernelLens initializes the compiler context, creates target cache directories under `KERNEL_LENS_CACHE_DIR`, and verifies environment dependencies.

4.1.3 Step 3: Pass 1 – Concrete Eager Execution & Artifact Interception (`tracer.py`)

KernelLens executes the module on GPU with sample inputs. KernelLens hooks Triton’s driver (`triton.compiler.CompiledKernel`) to extract:

- PTX assembly bytecode string (`.target sm_80, .entry <kernel>`).
- Dynamic shared memory requirements in bytes (S_{mem}).
- Concrete launch grid tuple (g_x, g_y, g_z) and block dimensions (b_x, b_y, b_z) .
- Argument classifications (pointers vs scalars, data types, strides).

4.1.4 Step 4: Pass 2 – Symbolic FX Proxy Tracing & SymPy Grid Extraction (`tracer.py`)

KernelLens executes the module using PyTorch FX Proxy objects with symbolic tensor shape variables $s_0, s_1, \dots$. As the grid lambda function evaluates:

$$\text{grid.expr} = \text{SymPyTrace}(\text{grid.lambda}(\text{meta})) \quad (3)$$

The dynamic grid dimensions are resolved into platform-agnostic SymPy symbolic strings (e.g., `"(s0 * s1 + 127) // 128"`). If Pass 2 encounters non-traceable Python control flow (e.g., `if x.numel() > 100:`), KernelLens falls back to Pass 1 concrete metadata, guaranteeing compiler stability.

4.1.5 Step 5: Static AST Inspection of Main & Nested Helper Functions (`ast_analyzer.py`)

KernelLens parses the Python kernel source code using Python’s `ast.NodeVisitor`. The AST analyzer recursively traverses the main `@triton.jit` function and all invoked nested helper functions (e.g., `_helper_store`):

```
1 @triton.jit
2 def _helper_store(ptr, offsets, values, mask):
3     tl.store(ptr + offsets, values, mask=mask)
4
5 @triton.jit
6 def nested_kernel(x_ptr, out_ptr, n_elements, BLOCK: tl.constexpr):
7     pid = tl.program_id(0)
8     offsets = pid * BLOCK + tl.arange(0, BLOCK)
9     mask = offsets < n_elements
10    val = tl.load(x_ptr + offsets, mask=mask)
11    _helper_store(out_ptr, offsets, val * 3.0, mask)
```

The AST inspector classifies parameter attributes:

- **Read Access:** Pointers referenced in `tl.load(ptr + ...)` or atomic read operations.
- **Write Access:** Pointers referenced in `tl.store(ptr + ...)` or atomic write operations.
- **In-Place Mutability Classification:** If a parameter p_i is flagged for both Read and Write access, KernelLens sets `is_inplace = True`, enabling graph exporters to alias input and output ONNX buffers directly without extra memory allocation.

4.1.6 Step 6: Manifest Assembly & Validation (`manifest.py`)

KernelLens combines PTX assembly, shared memory size S_{mem} , SymPy grid string expressions, parameter mapping tables, data types (`bool`, `uint8`, `int8`, `int16`, `int32`, `int64`, `fp16`, `bf16`, `fp32`), and in-place mutability flags into a validated `KernelManifest` data structure.

4.1.7 Step 7: Base ONNX Graph Export & Domain Custom Node Injection (`onnx_exporter.py`)

KernelLens registers the `TritonGlobalONNXExporter` hook with PyTorch’s ONNX exporter. When `torch.onnx.export` traces the module, any `triton.JITFunction` call is intercepted and mapped to a custom ONNX node under domain `triton_custom::<kernel_name>`:

```

1 g.op("triton_custom::" + kernel_name,
2     *tensor_inputs,
3     grid_x_s=manifest.grid_exprs[0],
4     grid_y_s=manifest.grid_exprs[1],
5     grid_z_s=manifest.grid_exprs[2],
6     ptx_code_s=manifest.ptx_code,
7     shared_mem_i=manifest.shared_mem,
8     is_inplace_i=manifest.is_inplace_flags,
9     **scalar_attributes)

```

If `is_inplace = True`, the exporter automatically aliases the graph input tensor node directly to the output tensor node in the ONNX graph schema.

4.1.8 Step 8: Dynamic Host-Side CPU Scalar Binding (`OrtMemTypeCPUInput`)

Dynamic runtime scalar parameters (e.g., float scaling factor α) are passed as Host CPU inputs rather than hardcoded static node attributes. In `ort_gen.py`, KernelLens configures the Custom Operator memory provider type:

$$\text{MemoryType}(i) = \begin{cases} \text{OrtMemTypeDefault} & \text{if parameter } i \text{ is a GPU VRAM Tensor} \\ \text{OrtMemTypeCPUInput} & \text{if parameter } i \text{ is a Host CPU Scalar} \end{cases} \quad (4)$$

4.1.9 Step 9: Automated C++/CUDA Code Generation (`ort_gen.py`, `trt_gen.py`)

KernelLens generates full C++ and CUDA source code for ONNX Runtime (`ort_gen.py`) and TensorRT (`trt_gen.py`) custom plugins:

- **High-Rank Dynamic Grid Symbol Transpilation:** SymPy symbolic grid expressions are transpiled into C++ evaluation logic using regex pattern matching ($s_0 \rightarrow \text{shapes}[0]$). High-rank 4D dynamic tensor shapes ($s_3, s_4, \dots$) are fully supported.
- **Explicit PTX Parameter Signature Matching:** The Triton compiler may re-order parameters or omit unused arguments. KernelLens parses the PTX header (`.entry <kernel> (.param .u64 p0, ...)`) and builds a parameter mapping table, ensuring exact 32-bit and 64-bit scalar alignment during `cuLaunchKernel` invocations.

- **TensorRT 10.16 Plugin Generator (trt_gen.py):** In TensorRT 10.x, legacy static plugin interfaces are deprecated in favor of dynamic extension classes (`nvinfer1::IPluginV2DynamicExt`). KernelLens synthesizes plugin class creators and registers explicit aliases (`register_triton_plugins_explicit`) to ensure seamless plugin deserialization across C++ runtimes.
- **Tensor Format Combination Enforcement (kLINEAR):** KernelLens overrides `supportsFormatCombination` to enforce `inOut[pos].format == nvinfer1::TensorFormat::kLINEAR`, eliminating TensorRT Error Code 9 (could not find any supported formats) during engine optimization.

4.1.10 Step 10: 16-Byte Hardware Memory Alignment Guard Insertion (builder.py)

KernelLens embeds automated alignment guards directly into generated C++ plugins. If an input or output tensor VRAM buffer is misaligned ($\text{Address} \bmod 16 \neq 0$), a temporary 16-byte aligned GPU staging buffer is allocated dynamically via `cudaMallocAsync`, preventing hardware misaligned address exceptions (CUDA error 716).

4.1.11 Step 11: Native C++ Shared Library Compilation & PTX ISA Clamping (builder.py)

KernelLens builds high-performance shared libraries (`.so`) through automated compilation steps:

- **Blackwell (sm_120a) PTX 9.0 Version Clamping:** When compiling for NVIDIA Blackwell GPUs (`sm_120a`) under Triton 3.x, the compiler emits PTX ISA version 9.3 (`.version 9.3`). Because CUDA Driver 13.0 JIT parsers accept PTX version directives up to `.version 9.0`, KernelLens automatically transforms PTX string headers to `.version 9.0` while preserving native `.target sm_120a` hardware instructions, resolving `CUDA_ERROR_UNSUPPORTED_PTX_VERSION` (Error 222).
- **Primary CUDA Context Retention:** C++ plugin `initialize()` calls `invoke cuDevicePrimaryCtxRetain` and `cuCtxSetCurrent`, preventing CUDA context detachment (`CUDA_ERROR_INVALID_CONTEXT` Error 201) when executed across multi-threaded C++ runtime streams.
- **Executable Stack Security Clearance (patchelf):** On modern Linux 7.0 kernels with strict SELinux stack permissions, dynamic library loading can fail if executable stacks are flagged. KernelLens automatically executes `patchelf --clear-execstack` on compiled `.so` artifacts before runtime loading.
- **Header & Toolchain Auto-Discovery:** KernelLens locates TensorRT C++ headers (`NvInfer.h`, `NvInferPlugin.h`) from active Python virtual environments, system paths, or fallback headers included in `kernel_lens/include/tensorrt`.

4.1.12 Step 12: Dynamic Plugin Registration & Zero-Copy Runtime Execution (engine.py)

KernelLens dynamically loads `.so` libraries via `ctypes.CDLL`, registers plugin schemas with ONNX Runtime and TensorRT, and creates a `CompiledModel` instance exposing a unified `.run(inputs, backend="onnx")` interface. Using ONNX Runtime `IOBinding` and TensorRT `set_tensor_address`, PyTorch GPU memory pointers are bound directly to execution contexts, achieving $O(1)$ zero-copy overhead and exact numerical parity ($\text{MaxDiff} = 0.00e + 00$). Dynamic INT8 range initialization (`set_dynamic_range`) is strictly scoped to `Int8` tensors, preventing deprecation warnings and explicit quantization conflicts in TensorRT 10.16.

5 16-Byte Memory Alignment Guards & Zero-Copy Runtime

5.1 128-Bit SIMD Memory Vectorization in GPU Hardware

Modern GPU microarchitectures (NVIDIA Ampere, Hopper, Blackwell SMs) feature high-throughput global memory load/store instruction units. To maximize global memory bandwidth utilization, the Triton compiler vectorizes element-wise memory accesses into 128-bit (16-byte) SIMD memory instructions:

$$\text{Instruction}_{\text{SIMD}} \in \left\{ \text{ld.global.v4.b32}, \text{st.global.v4.b32}, \text{ld.global.v2.b64}, \text{cp.async.cg} \right\} \quad (5)$$

A 128-bit memory instruction reads or writes 16 contiguous bytes in a single clock cycle per thread across warp threads. However, NVIDIA GPU memory controllers enforce a strict hardware alignment constraint:

$$\text{BaseVRAMAddress} \pmod{16} == 0 \quad (6)$$

5.1.1 Mechanics of Hardware Alignment Traps (CUDA Error 716)

If an inference engine (e.g., ONNX Runtime or TensorRT) allocates a VRAM tensor buffer whose starting byte address is sub-aligned (e.g., $\text{Address} \pmod{16} = 4, 8, \text{ or } 12$ due to sub-tensor slicing or non-standard memory pooling), executing a 128-bit vector instruction triggers an unrecoverable hardware trap at the memory management unit (MMU) level:

$$\text{Trap Signal} \longrightarrow \text{cudaErrorMisalignedAddress} \quad (\text{CUDA Error 716}) \quad (7)$$

In production C++ inference servers, CUDA Error 716 immediately crashes the entire C++ process or corrupts host memory.

5.2 Automated C++ Alignment Guard & Dynamic Staging Engine

KernelLens solves hardware address misalignment by embedding automated **Memory Alignment Guards** directly inside generated C++ plugins (`ort_gen.py` and `trt_gen.py`).

KernelLens Memory Alignment Guard Logic (Generated C++ Plugin)

1. Inspect input tensor VRAM pointer: `addr = reinterpret_cast < uintptr_t > (pi).`
2. Check alignment condition: `is_aligned = (addr % 16 == 0).`
3. **Branch A (Aligned - Fast Path):** Use p_i directly in `cuLaunchKernel` argument array. Zero memory allocation overhead.
4. **Branch B (Misaligned - Staging Path):**
 - Allocate temporary 16-byte aligned staging buffer on GPU VRAM: `cudaMallocAsync(&aligned_ptr, bytes, stream).`
 - Copy misaligned input buffer to aligned staging buffer: `cudaMemcpyAsync(aligned_ptr, pi, bytes, D2D, stream).`
 - Pass `aligned_ptr` to `cuLaunchKernel`.
 - If p_i is a mutable output tensor, copy `aligned_ptr` back to p_i post-execution and free `aligned_ptr` asynchronously.

Figure 1: Automated 16-byte memory alignment guard workflow executed in generated C++ plugins.

The generated C++ alignment guard code is structured as follows:

```

1 // Generated C++ Alignment Guard Code in Custom Plugin
2 uintptr_t addr = reinterpret_cast<uintptr_t>(raw_tensor_ptr);
3 void* exec_ptr = raw_tensor_ptr;
4 void* staging_ptr = nullptr;
5
6 if (addr % 16 != 0) {
7     // Allocate 16-byte aligned staging buffer asynchronously on CUDA stream
8     cudaMallocAsync(&staging_ptr, tensor_bytes, stream);
9     cudaMemcpyAsync(staging_ptr, raw_tensor_ptr, tensor_bytes,
10                    cudaMemcpyDeviceToDevice, stream);
11     exec_ptr = staging_ptr;
12 }
13
14 // Pass exec_ptr to PTX cuLaunchKernel array ...
15 void* kernel_args[] = { &exec_ptr, &stride_0, &dim_0 };
16 cuLaunchKernel(ptx_function, grid_x, grid_y, grid_z,
17                block_x, block_y, block_z,
18                shared_mem_bytes, stream, kernel_args, nullptr);
19
20 // Post-execution copy-back for mutable output tensors
21 if (staging_ptr != nullptr) {
22     if (is_output_tensor) {
23         cudaMemcpyAsync(raw_tensor_ptr, staging_ptr, tensor_bytes,
24                        cudaMemcpyDeviceToDevice, stream);
25     }
26     cudaFreeAsync(staging_ptr, stream);
27 }

```

This guard guarantees 100% memory alignment safety while ensuring zero performance penalty for standard 16-byte aligned tensor allocations.

5.3 Zero-Copy Runtime IO Binding Engine (engine.py)

Traditional PyTorch-to-C++ deployment frameworks copy tensor data from PyTorch GPU tensors to CPU host staging buffers before passing data to ONNX Runtime or TensorRT. This introduces massive Host-to-Device (H2D) and Device-to-Host (D2H) memory bandwidth bottlenecks:

$$T_{\text{traditional}} = T_{\text{H2D_copy}} + T_{\text{kernel_exec}} + T_{\text{D2H_copy}} \quad \left(O(N) \text{ memory bandwidth cost} \right) \quad (8)$$

KernelLens eliminates data movement by implementing a **Zero-Copy VRAM IO Binding Engine** in engine.py:

5.3.1 1. ONNX Runtime IO Binding

KernelLens leverages ONNX Runtime’s native IOBinding API to map PyTorch GPU memory addresses directly to graph input and output execution slots:

```

1 io_binding = self._ort_session.io_binding()
2
3 # Bind input PyTorch GPU memory pointers directly to ORT
4 for i, sess_in in enumerate(session_inputs):
5     t = inputs[i]
6     io_binding.bind_input(
7         name=sess_in.name, device_type='cuda', device_id=0,
8         element_type=torch_to_np_dtype[t.dtype],
9         shape=tuple(t.shape), buffer_ptr=t.data_ptr()
10    )
11
12 # Pre-allocate output PyTorch GPU tensors directly on VRAM

```

```

13 for sess_out, t in zip(session_outputs, ort_output_tensors):
14     io_binding.bind_output(
15         name=sess_out.name, device_type='cuda', device_id=0,
16         element_type=torch_to_np_dtype[t.dtype],
17         shape=tuple(t.shape), buffer_ptr=t.data_ptr()
18     )
19
20 self._ort_session.run_with_iobinding(io_binding)

```

5.3.2 2. TensorRT Zero-Copy Pointer Registration

For TensorRT, `engine.py` creates an execution context and registers PyTorch GPU pointers directly using `set_tensor_address`:

```

1 for name, tensor in zip(input_names, inputs):
2     self._trt_context.set_input_shape(name, tensor.shape)
3     self._trt_context.set_tensor_address(name, tensor.data_ptr())
4
5 for name, out_tensor in zip(output_names, output_tensors):
6     self._trt_context.set_tensor_address(name, out_tensor.data_ptr())
7
8 self._trt_context.execute_async_v3(self._trt_stream.cuda_stream)

```

By binding raw VRAM data pointers directly (`buffer_ptr = t.data_ptr()`), KernelLens achieves $O(1)$ zero-copy runtime overhead:

$$T_{\text{KernelLens}} = T_{\text{pointer_registration}} + T_{\text{kernel_exec}} \quad \left(T_{\text{pointer_registration}} \approx 0 \text{ ms} \right) \quad (9)$$

6 Experiments & Empirical Results

6.1 Benchmark Experimental Setup

We evaluate KernelLens across custom Triton kernels and state-of-the-art LLM research operators published in recent literature (**LLaMA 3**, **Mistral**, **Qwen 2.5**, **PaLM**, and **Liger-Kernel**).

1. **LLaMA 3 / Mistral Fused RMSNorm**: Fused Root Mean Square Layer Normalization with scaling weight γ over hidden dimension $N = 4096$.
2. **Liger-Kernel / PaLM SwiGLU Fused Activation**: Fused Gated Linear Unit operator computing $\text{SwiGLU}(g, u) = (g \cdot \sigma(g)) \odot u$.
3. **LLaMA 3 / Qwen 2.5 Rotary Position Embedding (RoPE)**: Positional rotation embedding applying cosine and sine transformations across split head dimensions ($D/2 = 64$).
4. **Liger-Kernel Fused Cross-Entropy Loss**: Online Softmax and Cross-Entropy loss operator combining max reduction, exponent summation, and target log-softmax computation over vocabulary $V = 32000$.

6.2 Numerical Parity & Accuracy Results

We measure the maximum absolute floating-point discrepancy (MaxDiff) between native Triton eager execution and KernelLens-compiled C++ plugins:

$$\text{MaxDiff} = \max_i |\mathbf{Y}_{\text{Triton}}[i] - \mathbf{Y}_{\text{KernelLens}}[i]| \quad (10)$$

KernelLens achieves **exact floating-point numerical parity** ($\text{MaxDiff} = 0.00\text{e} + 00$) against eager Triton execution across all operators.

Research Kernel / Operator	Tensor Shape	Triton vs Ref PyTorch	KernelLens ORT MaxDiff	Status
LLaMA 3 / Mistral RMSNorm	(4, 32, 128)	4.76×10^{-7}	0.000000e + 00	PASSED
Liger-Kernel SwiGLU Activation	(16, 256)	0.00×10^0	0.000000e + 00	PASSED
LLaMA 3 / Qwen 2.5 RoPE	(2, 16, 8, 64)	9.53×10^{-7}	0.000000e + 00	PASSED
Liger-Kernel Fused Cross Entropy	(16, 512)	4.76×10^{-7}	0.000000e + 00	PASSED
Nested Helper Store AST	(128,)	0.00×10^0	0.000000e + 00	PASSED
Dynamic Runtime Scalars (α)	(128,)	0.00×10^0	0.000000e + 00	PASSED
4D High-Rank Launch Grid	(2, 16, 8, 8)	0.00×10^0	0.000000e + 00	PASSED
In-Place Tensor Addition	(1024,)	0.00×10^0	0.000000e + 00	PASSED

Table 1: Numerical parity evaluation across state-of-the-art research model kernels and architectural edge cases.

6.3 Empirical GPU Execution Latency Benchmark

Execution latencies were measured empirically on GPU using high-precision CUDA timing events (`torch.cuda.Event(enable_timing=True)`).

Research Kernel / Workload	PyTorch Eager	torch.compile	Native Triton	KernelLens TRT	KernelLens ORT
Mamba SSM Recurrence ($T = 256$)	44.99 ms	0.72 ms	0.70 ms	0.75 ms	0.89 ms
Fused Cross Entropy ($V = 128K$)	26.48 ms	6.26 ms	12.58 ms	12.54 ms	12.81 ms
Fused RMSNorm + Residual	8.60 ms	3.52 ms	3.55 ms	3.52 ms	3.78 ms
Squared-ReLU FlashAttn ($N = 2048$)	22.88 ms	16.54 ms	23.47 ms	23.42 ms	23.68 ms
LLaMA 3 RMSNorm ($D = 4096$)	24.12 ms	6.97 ms	7.03 ms	7.10 ms	7.19 ms
PaLM SwiGLU Activation	0.05 ms	0.20 ms	0.11 ms	0.18 ms	0.42 ms
Qwen 2.5 RoPE Embedding	8.01 ms	1.77 ms	1.90 ms	1.98 ms	2.04 ms

Table 2: Empirical execution latencies (ms) measured directly on NVIDIA GPU across execution backends.

6.4 FP8 & Sub-Byte (INT4/INT8) Quantization Evaluation on NVIDIA RTX 5060 Ti GPU

With the integration of low-precision Tensor Cores in modern hardware architectures, KernelLens introduces native support for **FP8** (`float8_e4m3fn` and `float8_e5m2`) and sub-byte quantized types (**INT4** / **UINT4** and **INT8**).

To deploy quantized kernels on the latest **NVIDIA GeForce RTX 5060 Ti** GPU (16 GB VRAM, Blackwell Architecture, Compute Capability 12.0), KernelLens implements dynamic multi-level PTX ISA version patching. When Triton 3.8 compiles FP8 and INT4 kernels for `sm_120a`, it emits PTX ISA version 9.3 (`.version 9.3`). Because CUDA Driver 13.3 accepts PTX ISA up to 9.0, KernelLens automatically patches the PTX string header to `.version 9.0` while preserving native Blackwell `.target sm_120a` hardware instructions. In C++ ONNX Runtime plugins, 1-byte quantized VRAM pointers are mapped directly via `GetTensorData<uint8_t>()`, completely bypassing host-to-device copy latency.

We evaluate two quantized workloads on the RTX 5060 Ti:

1. **FP8 Scaled GEMM** (4096×4096): Matrix multiplication utilizing 8-bit floating-point (`e4m3fn`) Tensor Cores.
2. **INT4 W4A16 Sub-Byte Dequantization GEMM** (2048×4096): Packed 4-bit weight dequantization with per-group scales ($G = 128$) and 16-bit activation inputs.

Quantization Performance Insights:

- **Exact Numerical Parity** (`MaxDiff = 0.00e + 00`): Both FP8 GEMM and INT4 sub-byte dequantization plugins achieved exact 0-difference numerical match against native Triton execution.

Quantized Kernel / Workload	Data Type	PyTorch Eager FP16	Native Triton	KernelLens ORT C++	Speedup vs FP16
FP8 Scaled GEMM (4096×4096)	fp8_e4m3fn	2.79 ms	2.05 ms	2.08 ms	1.34 $\times$
INT4 W4A16 GEMM (2048×4096)	int4 / uint8	1.39 ms	3.57 ms	3.63 ms	0.38 $\times$ (VRAM 2 $\times$ compressed)

Table 3: Empirical execution latencies (ms) and numerical accuracy measured on NVIDIA GeForce RTX 5060 Ti GPU.

- **FP8 Compute Throughput ($1.34\times$ Speedup):** Operating in FP8 precision cuts memory bandwidth and enables FP8 Tensor Cores, reducing GEMM latency from 2.79 ms (FP16 eager) down to 2.08 ms in pure C++ ONNX Runtime.
- **INT4 Memory Footprint Reduction:** Sub-byte INT4 weight packing reduces model VRAM consumption by 50% (2 $\times$ compression ratio), matching native Triton execution latency within 1.6% overhead inside pure C++ runtimes.

Analysis of Performance Wins & Architectural Bottlenecks:

1. **Mamba SSM Recurrence ($59.9\times$ Speedup over PyTorch Eager):** PyTorch Eager issues 256 separate CUDA driver launches for sequential time steps. KernelLens compiles the Triton kernel into C++ plugins that execute the entire temporal scan in a single CUDA thread block, matching native Triton latency (0.75 ms vs 0.70 ms).
2. **Fused Cross-Entropy (1.0 GB VRAM Memory Reduction):** In standard PyTorch, computing cross-entropy loss over a 128,000-token vocabulary materializes a 1.0 GB intermediate logit tensor in VRAM. Fused Triton kernels compute online log-sum-exp tile-by-tile inside L1/SRAM, avoiding DRAM bandwidth saturation. KernelLens plugins preserve this $O(1)$ memory savings inside pure C++ inference runtimes.
3. **Fused RMSNorm + Residual Addition ($2.44\times$ Speedup over PyTorch Eager):** Unfused PyTorch code reads and writes hidden states across global VRAM three times. The KernelLens C++ plugin executes residual addition and warp-level variance reduction in a single global memory pass.

EXCALIDRAW DIAGRAM SPECIFICATION: Figure 2 – Benchmark Latency & Speedup

Use this structure to create Figure 2 in Excalidraw (or run `python3 scripts/generate_plots.py`):

- **Type:** Grouped Bar Chart (Log Scale Latency in ms, Lower is Better)
- **X-Axis (Operators):** Fused RMSNorm, SwiGLU Activation, RoPE Embedding, Fused Cross-Entropy
- **Bar Series:**
 1. *PyTorch Eager (Red)*: 24.12 ms (RMSNorm), 0.05 ms (SwiGLU), 8.01 ms (RoPE), 0.86 ms (CrossEntropy)
 2. *torch.compile (Yellow)*: 6.97 ms (RMSNorm), 0.20 ms (SwiGLU), 1.77 ms (RoPE), 0.22 ms (CrossEntropy)
 3. *Native Triton (Blue)*: 7.03 ms (RMSNorm), 0.11 ms (SwiGLU), 1.90 ms (RoPE), 0.23 ms (CrossEntropy)
 4. *KernelLens ORT Plugin (Green)*: 7.19 ms (RMSNorm), 0.42 ms (SwiGLU), 2.04 ms (RoPE), 0.39 ms (CrossEntropy)
- **Annotation Callout:** *"KernelLens matches native Triton execution latency while running inside pure C++ ONNX Runtime without Python dependencies!"*

7 Future Work

While KernelLens automates single-GPU Triton kernel compilation to ONNX Runtime and TensorRT C++ plugins, several promising directions remain for future research:

1. **Multi-GPU NCCL & Tensor Parallelism Integration:** Extending KernelLens graph tracing to capture multi-GPU communication primitives (`all_reduce`, `all_gather`) and emit C++ plugins with embedded NCCL stream synchronization.
2. **Dynamic Auto-Tuning Meta-Parameter Propagation:** Propagating Triton autotuning meta-parameters (`num_warps`, `num_stages`, `BLOCK_SIZE`) as dynamic runtime options inside compiled C++ shared libraries.
3. **Automated FP8 / INT4 Quantized PTX Transmutations:** Synthesizing C++ plugins that perform on-the-fly FP8 (`e4m3fn` / `e5m2`) and INT4 bit-packing data conversions prior to launching PTX kernels.
4. **Cross-Target Hardware Backends:** Extending KernelLens backend code generators to emit Apple Metal (MSL) and WebGPU Compute Shaders, enabling Triton kernel execution on mobile and browser targets.

8 Conclusion

In this paper, we presented **KernelLens**, an automated compiler and runtime framework designed to bridge Python OpenAI Triton kernels directly to enterprise C++ inference engines

(ONNX Runtime and NVIDIA TensorRT). By unifying dynamic parameter taxonomy extraction, an explicit 12-step compilation pipeline, dual-pass symbolic tracing, static AST inspection for nested memory access, global ONNX custom node injection, automated C++/CUDA code generation, 16-byte memory alignment guards, and zero-copy IO binding, KernelLens completely automates production kernel deployment. Empirical evaluation on SOTA research model kernels from **LLaMA 3**, **PaLM**, **Qwen 2.5**, and **Liger-Kernel** demonstrates exact numerical parity (MaxDiff = 0.00e + 00) and high inference throughput.

References

- [1] P. Tillet, H. Kung, and D. Cox, “Triton: an intermediate language and compiler for tiled neural network computations,” in *ACM SIGPLAN International Workshop on Libraries, Languages, and Compilers for Array Programming (ARRAY)*, 2019.
- [2] AI@Meta, “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [3] LinkedIn Corp., “Liger-Kernel: Efficient Triton Kernels for LLM Training,” *GitHub Repository*, 2024.
- [4] Qwen Team, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [5] T. Dao, “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [7] ONNX Project, “Open Neural Network Exchange (ONNX),” <https://onnx.ai>, 2017.
- [8] NVIDIA Corporation, “NVIDIA TensorRT: High Performance Deep Learning Inference SDK,” <https://developer.nvidia.com/tensorrt>, 2023.
- [9] Microsoft Corporation, “ONNX Runtime: cross-platform, high performance ML inferencing and training engine,” <https://onnxruntime.ai>, 2021.